Malicious URL Detection Using Machine Learning

Objective

The goal of this project is to build a robust model that can automatically detect whether a URL is **malicious** or **benign**. This is an essential task in cybersecurity as malicious URLs are often used in phishing, malware distribution, and social engineering attacks.

Step 1: Data Preprocessing

We started by loading and filtering a dataset containing labeled URLs. Each URL was tagged as one of multiple categories (e.g., benign, phishing, malware, defacement), which we later encoded as numerical labels.

```
df = df[['url', 'type', 'type_code']] # type_code --> (0,1,2,3)
dataset = Dataset.from_pandas(df, preserve_index=False)
```

Step 2: Feature Engineering (Core of the Detection)

The **intelligence** of this model comes from the **features** we extract from URLs. These features attempt to capture patterns commonly found in malicious links. Here's a breakdown of the engineered features and why they matter:

1. `URL length:`
   - **What**: Total number of characters in the URL.
   - **Why**: Malicious URLs are often longer than normal to obscure malicious components or mimic legitimate URLs.

2. `Number of "@":`
   - **What**: Number of `@` symbols in the URL.
   - **Why**: Attackers use `@` to hide real domains. Everything before `@` is ignored by browsers.

3. `Number of "-":`
   - **What**: Number of hyphens.
   - **Why**: Malicious domains often include hyphens to impersonate legitimate sites (e.g., `pay-pal.com`).

4. `Number of ".":`
   - **What**: Number of dots.
   - **Why**: URLs with many subdomains may be suspicious (e.g., `bank.algeria.dz.national.xyz`).

5. `Number of "=":`
   - **What**: Number of equal signs.
   - **Why**: Used heavily in phishing URLs to pass suspicious parameters.

6. `Number of digits:`
   - **What**: Count of digits in the URL.
   - **Why**: Fake URLs often use numbers to resemble legitimate IDs or confuse users.

7. `Number of letters:`
   - **What**: Count of alphabetic characters.
   - **Why**: Short URLs with few letters and many symbols can be suspicious.

8. `Number of "?":`
   - **What**: Number of `?` characters.
   - **Why**: Attackers often craft URLs with many query parameters to track victims or trigger behaviors.

9. `Number of "%":`
   - **What**: Percent signs are used in encoding.
   - **Why**: Too many encodings may indicate URL obfuscation.

10. `Number of "http" and "https":`

- **What**: Count of "http" or "https" inside the URL.
- **Why**: Duplicate or misplaced protocols can be part of deception (e.g., `http://https.malcious.com`).

11. `Number of "www":`

- **What**: Count of "www" substrings.
- **Why**: Excessive use may indicate fake domain nesting.

12. `Hostname length:`

- **What**: Number of characters in the domain name.
- **Why**: Long hostnames may be used to mimic legitimate URLs.

These features were added using `.apply(lambda ...)` operations across the dataset.

Step 3: Model Building with XGBoost

We used the **XGBoost** classifier for training:

```
from xgboost import XGBClassifier
```

Why XGBoost?

- **High performance**: It's an ensemble model based on boosting and often wins Kaggle competitions.
- **Handles non-linearity**: It can capture complex patterns without explicit feature transformation.
- **Robust to overfitting**: With proper regularization and tuning.

Step 4: Training and Evaluation

The dataset was split into:

- **70% training**
- **20% validation**
- **10% testing**

We trained the XGBoost model using:

```
model = XGBClassifier()
model.fit(X_train, y_train)
```

Then evaluated it using accuracy, precision, recall, and F1-score. The model achieved **high accuracy (~95%)** in distinguishing between normal and malicious URLs.

Final Results

Metric	Score
Accuracy	95%
Precision	93%
Recall	90%
F1-score	92%

Security Relevance

The strength of this system lies not just in AI, but in **understanding URL structure** and attacker behavior. By embedding security intuition into feature design, we built a model that can generalize well to real-world threats.

Conclusion

This project demonstrates how feature-rich URL analysis, combined with a powerful model like XGBoost, can effectively detect malicious activity. It's a practical application of AI in **cybersecurity**, where domain knowledge is as important as the algorithms used.